Deployment Guide

Local Setup • Cloud Deployment • Security • Multi-Tenant

Version 1.0 • August 2026

Table of Contents

1. Local Development Setup

Prerequisites

Requirement	Minimum	Recommended
Docker Desktop	4.0+	4.20+ with Compose v2
RAM	4 GB	8 GB
Disk Space	10 GB free	20 GB free
OS	Windows 10+, macOS 12+, Ubuntu 20+	Windows 11, Ubuntu 22.04

Windows Quick Start

Step 1: Double-click **setup.bat** (one-time only). This script:

→ Opens Docker Desktop if not running
→ Creates .env file with random SECRET_KEY and POSTGRES_PASSWORD
→ Builds Docker images for all 4 services
→ Installs Python test requirements
→ Runs the full test suite (110 tests) to verify the setup

Step 2: Double-click **start.bat**. This script:

→ Starts Docker Compose (db, redis, web, frontend)
→ Waits for PostgreSQL to be healthy
→ Runs Alembic migrations to create/update tables
→ Seeds demo data (company, admin user, sample items)
→ Opens browser to http://localhost:9009/

Linux / Any Server

One command does everything:

```
./start.sh
```

The start.sh script is idempotent — safe to run multiple times. It creates .env on first run, builds images, starts services, runs migrations, and seeds data.

Manual Steps (if needed)

```
# Build and start all services
docker compose up --build -d

# Watch logs
docker compose logs -f web

# Run migrations manually
docker compose exec -T web alembic upgrade head
```

```
# Seed demo data
docker compose exec -T web python -m scripts.seed

# Stop services (keeps data)
docker compose down

# Stop and DELETE all data
docker compose down -v
```

Demo Credentials

Field	Value
URL	http://localhost:9009/
Email	admin@example.com
Password	admin123
Company	DEMO
Swagger Docs	http://localhost:9009/api/v1/docs
Health Check	http://localhost:9009/health

2. Production Server Deployment

Server Requirements

Provider	Instance	Specs	Monthly Cost
DigitalOcean	Basic Droplet	4 vCPU, 8GB RAM, 100GB SSD	~$40 (1,400 EGP)
Hetzner Cloud	CPX41	8 vCPU, 16GB RAM, 240GB SSD	~$35 (1,225 EGP)
AWS EC2	t3.large	2 vCPU, 8GB RAM, 100GB EBS	~$60 (2,100 EGP)
Vultr	Cloud Compute	4 vCPU, 8GB RAM, 200GB SSD	~$48 (1,680 EGP)
Local VPS (Egypt)	Various	4 vCPU, 8GB RAM, 100GB SSD	1,500-3,000 EGP

Step-by-Step Deployment

Step 1: Install Docker

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install Docker
curl -fsSL https://get.docker.com | sh

# Add user to docker group
sudo usermod -aG docker $USER
newgrp docker

# Verify
docker --version # Should show 24+
```

Step 2: Copy Project

```
# Clone or copy project
scp -r ./ERP_System user@server:/opt/erp
ssh user@server
cd /opt/erp
```

Step 3: Configure Environment

```
# Copy template
cp .env.example .env

# Generate secret key
python3 -c "import secrets; print(secrets.token_hex(32))"

# Edit .env
nano .env

# Required changes:
```

```
# SECRET_KEY=
# POSTGRES_PASSWORD=<your-password>
# APP_ENV=production
# DEBUG=false
```

Step 4: Start Services

```
# Build and start
docker compose up --build -d

# Wait for health check
curl http://localhost:9009/health
# Should return: {"status": "ok", ...}
```

Step 5: Create Admin User

```
# Seed demo data
docker compose exec -T web python -m scripts.seed

# Or create custom admin
docker compose exec -T web python -m scripts.seed
--admin-email admin@yourcompany.com
--admin-password 'YourSecurePassword!'
```

Step 6: Set Up Backups

```
# Make backup script executable
chmod +x scripts/backup_db.sh

# Add to crontab (daily at 2 AM)
crontab -e
0 2 * * * cd /opt/erp && ./scripts/backup_db.sh >> backups/backup.log 2>&1
```

3. Domain & SSL Configuration

DNS Setup

Create a DNS A record pointing your domain to your server's public IP address:

```
Type: A | Name: erp | Value: YOUR_SERVER_IP | TTL: 3600
```

This creates erp.yourdomain.com pointing to your server. DNS propagation may take 5 minutes to 48 hours.

Option A: Caddy (Recommended)

Caddy automatically obtains and renews SSL certificates via Let's Encrypt. Zero configuration required for HTTPS.

```
# Install Caddy
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' |
sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' |
sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt update && sudo apt install caddy

# Configure
sudo nano /etc/caddy/Caddyfile

# Content:
erp.yourdomain.com {
reverse_proxy localhost:9009
}

# Restart
sudo systemctl restart caddy
sudo systemctl enable caddy
```

Option B: Nginx + Certbot

```
# Install Nginx and Certbot
sudo apt install nginx certbot python3-certbot-nginx

# Create Nginx config
sudo nano /etc/nginx/sites-available/erp

# Content:
server {
listen 80;
server_name erp.yourdomain.com;

location /api/ {
proxy_pass http://127.0.0.1:9009;
proxy_set_header Host $host;
```

```
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
}

location / {
proxy_pass http://127.0.0.1:9009;
proxy_set_header Host $host;
}
}

# Enable and get SSL
sudo ln -s /etc/nginx/sites-available/erp /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d erp.yourdomain.com
```

4. Multi-Tenant Architecture

ERP Pro uses a Shared Database architecture — all companies reside in the same PostgreSQL database, isolated by company_id foreign keys on every business table. This is the most cost-effective and operationally simple model for SaaS deployments.

What Each Company Gets

Resource	Scope	Isolation Method
Company Profile	Per company	Unique id, name, code, settings
Branches	Per company	Branch records linked by company_id
Users	Shared identity	user_roles links users to companies with roles
Roles & Permissions	Per company	Roles and permissions scoped to company_id
Items & Categories	Per company	All item tables have company_id FK
Partners	Per company	partners.company_id FK
Warehouses & Stock	Per company	warehouse_stock.company_id FK
Invoices	Per company	sales_invoices.company_id FK
Payments	Per company	payments.company_id FK
Journal Entries	Per company	journal_entries.company_id FK
Numbering Sequences	Per company	Numbering is sequential per company
Module Settings	Per company	company_settings.enabled_modules

Data Isolation Enforcement

Every API request that touches business data passes through the get_current_company_id() dependency. This reads the current_company_id from the user's auth session and returns it to the route handler. All subsequent queries filter by this company_id.

If a user tries to access data from a company they don't have a role in, the system returns HTTP 403 Forbidden. If no company is selected (session not scoped), the system returns HTTP 409 Conflict.

Superuser Access

Users with is_superuser=True bypass all company-scoping and permission checks. They can access the Platform admin panel to manage all tenants, create companies, reset passwords, and view all data. Use sparingly and only for system administrators.

5. Adding New Companies

Method 1: SuperAdmin Panel (Recommended)

1. Login as superuser (admin@example.com / admin123)
2. Navigate to SuperAdmin in the sidebar
3. Click 'Create Company'
4. Fill in: Company Name, Code, Base Currency (EGP), Activity Type
5. Enable desired modules (Sales, Purchases, Inventory, etc.)
6. System auto-creates: Company + Main Branch + Admin Role + Admin User
7. Share the admin credentials with the company owner

Method 2: API Direct

```
POST /api/v1/platform/companies
Authorization: Bearer <superuser_token>
Content-Type: application/json

{
"name": "Acme Trading Co",
"code": "ACME",
"base_currency": "EGP",
"activity_type": "trading",
"admin_email": "admin@acme.com",
"admin_password": "SecureP@ss2026!",
"enabled_modules": ["sales", "purchases", "inventory", "accounting", "hr"]
}
```

Method 3: Seed Script

```
# Create a new company via seed script
docker compose exec -T web python -m scripts.seed
--company-code ACME
--company-name 'Acme Trading'
--admin-email admin@acme.com
--admin-password 'SecureP@ss2026!'
```

What Gets Created Automatically

Resource	Details
Company	Name, code, base_currency, activity_type, is_active=True
Main Branch	Code: MAIN, name: Main Branch
Company Settings	All modules enabled, cost_method: weighted_average
Admin Role	Name: Admin, all 63 permissions granted
Admin User	Email + password, is_superuser for platform access

Resource	Details
User Role	Links admin user to company with Admin role

6. Security Hardening

6.1 Change Default Secrets

```
# Generate new SECRET_KEY
python3 -c "import secrets; print(secrets.token_hex(32))"

# Update .env
SECRET_KEY=<new_key>

# Restart
docker compose restart web

WARNING: This invalidates all existing tokens. All users must re-login.
```

6.2 Change Database Password

```
# Generate strong password
python3 -c "import secrets; print(secrets.token_urlsafe(20))"

# Update .env
POSTGRES_PASSWORD=<new_password>

# WARNING: This requires recreating the database
# Backup first: ./scripts/backup_db.sh
docker compose down -v
docker compose up --build -d
docker compose exec -T web python -m scripts.seed
```

6.3 Firewall Rules

```
# Enable UFW
sudo ufw enable

# Allow only necessary ports
sudo ufw allow 22/tcp # SSH
sudo ufw allow 80/tcp # HTTP (redirects to HTTPS)
sudo ufw allow 443/tcp # HTTPS

# Block everything else
sudo ufw default deny incoming
sudo ufw default allow outgoing

# NEVER expose PostgreSQL (5432) or Redis (6379) to the internet
```

6.4 Disable Debug Mode

```
# In .env
DEBUG=false
APP_ENV=production

# This disables: Swagger docs in production,
# verbose error messages, and development features
```

6.5 Regular Updates

```
# Update Docker images monthly
docker compose pull
docker compose up --build -d

# Update system packages
sudo apt update && sudo apt upgrade -y
```

7. Backup & Disaster Recovery

Automated Backup

```
# Manual backup
./scripts/backup_db.sh

# Automated daily backup (cron)
crontab -e
0 2 * * * cd /opt/erp && ./scripts/backup_db.sh >> backups/backup.log 2>&1

# Backup to custom directory
./scripts/backup_db.sh /var/backups/erp
```

Backups are stored as compressed SQL dumps in the backups/ directory. The script automatically keeps only the newest 14 files to manage disk usage.

Restore Procedure

```
# WARNING: This drops the existing database
./scripts/restore_db.sh backups/erp_erp_20260810_120000.sql.gz

# Or manually
gunzip -c backups/erp_erp_20260810_120000.sql.gz |
docker compose exec -T db psql -U erp -d erp
```

Always test your restore procedure before going live. A backup that has never been tested is not a backup.

Disaster Recovery Plan

Metric	Target	Method
RPO (Recovery Point)	24 hours	Daily automated backups
RTO (Recovery Time)	1 hour	Docker compose up from backup
Backup Retention	14 days	Automatic pruning in backup script
Off-site Backup	Weekly	Copy to S3 or alternative storage
Restore Test	Monthly	Run restore on staging environment

8. Monitoring & Maintenance

Health Check

```
curl http://localhost:9009/health
# Returns: {"status": "ok", "app": "ERP System", "environment": "production",
# "dependencies": {"database": "", "redis": ""}}
```

Service Status

```
docker compose ps # Show running containers
docker compose logs web # Backend logs
docker compose logs frontend # Frontend/Nginx logs
docker compose top # Process info
```

Database Maintenance

```
# Connect to database
docker compose exec db psql -U erp -d erp

# Check table sizes
SELECT pg_size_pretty(pg_total_relation_size tablename)
FROM pg_tables WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(tablename) DESC;

# Run VACUUM ANALYZE monthly
docker compose exec db psql -U erp -d erp -c 'VACUUM ANALYZE;'
```

9. Troubleshooting

Problem	Cause	Solution
Container won't start	Port conflict	Change ports in docker-compose.yml or stop conflicting service
Database connection refused	DB not ready	Wait 30s or: docker compose restart web
Migration errors	Schema drift	docker compose exec db psql -U erp -d erp -c 'DROP SCHEMA public CASCADE; CREATE SCHEMA public;' then re-seed
Frontend 502 Bad Gateway	Backend not running	docker compose logs web to check errors
Rate limit errors	Redis issue	Rate limiting degrades gracefully — check Redis: docker compose logs redis
Token expired	Session timeout	User must re-login (default: 24h token lifetime)
Permission denied (403)	Missing role	Assign appropriate role via SuperAdmin or Roles page
Module not accessible (403)	Module disabled	Enable module in Company Settings or via API
Disk space full	Backup accumulation	Backups auto-prune to 14; check: du -sh backups/